

- 1 What are Starter dependencies and what does “opinionated setup” mean?
- 2 What is Spring Boot Auto-Configuration and how does it work?
- 3 What are `application.properties` and `application.yml` used for?
- 4 What are `CommandLineRunner` and `ApplicationRunner` used for?

1 Why Spring Boot is Different

- **Automatic JSON input/output** → Spring Boot uses **Jackson** automatically with `spring-boot-starter-web`, so you don't manually serialize/deserialize objects.
- **Embedded Server** → Comes with **Tomcat/Jetty/Undertow** embedded. You don't need to deploy a WAR separately.
- **Opinionated Framework** → Provides **default best practices** so you don't need to configure everything manually.
- **Microservices Ready** → Works well with small, independent services.

2 Core Annotations

- `@SpringBootApplication` → Combines:
 1. `@Configuration`
 2. `@EnableAutoConfiguration` → Auto-configures beans based on dependencies
 3. `@ComponentScan` → Scans controllers, services, repositories automatically

- **@RestController / @Service / @Repository** → Mark classes as Spring-managed beans.
- **Auto-configuration** → Spring Boot decides default beans and configurations for your app.

1 Starter Dependencies & Opinionated Setup

Question: What are Starter dependencies and what does “opinionated setup” mean?

Answer:

- **Spring Boot Starters** are **pre-configured dependency bundles** that simplify project setup.
- Instead of adding multiple libraries manually, you include **one starter**, and it brings all required dependencies.
- **Opinionated Setup** → Spring Boot provides **default configurations based on best practices, reducing the need for manual setup.**

Examples:

Starter	Includes	Use case
<code>spring-boot-starter-web</code>	Spring MVC + Embedded Tomcat + Jackson	Build web apps with JSON support
<code>spring-boot-starter-data-jpa</code>	Hibernate + JPA APIs + DB support	Database operations
<code>spring-boot-starter-security</code>	Basic login + security filters	Authentication & authorization
<code>spring-boot-starter-test</code>	JUnit + Mockito + Spring Test	Testing

Interview Tip:

“Starters work with auto-configuration, automatically configuring beans based on classpath dependencies.”

2 Auto-Configuration

Question: What is Spring Boot Auto-Configuration and how does it work?

Answer:

- Automatically configures beans based on **dependencies present in the classpath.**
- **Eliminates manual configuration.**

How it works internally:

- Uses **@EnableAutoConfiguration + @Conditional** annotations.
- Example:
 - MySQL dependency present → MySQL DataSource auto-configured
 - Hibernate present → JPA auto-configured

Practical Benefit:

- Add `spring-boot-starter-data-jpa` → Hibernate, JPA, DB all auto-setup.

3 application.properties / application.yml

Question: What are `application.properties` and `application.yml` used for?

Answer:

- Configuration files in Spring Boot for **application settings:**
 - **Server port**
 - Database connections
 - Logging levels
 - Security configs

Examples:

```
server.port=8081
spring.application.name=my-app
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=1234
spring.jpa.hibernate.ddl-auto=update
```

```
server:
  port: 8081
spring:
  application:
    name: my-app
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
```

password: 1234
jpa:
hibernate:
ddl-auto: update

Interview Tip:

“They externalize configuration so code remains clean and can be changed without modifying code.”

4 CommandLineRunner & ApplicationRunner

Question: What are `CommandLineRunner` and `ApplicationRunner` used for?

Answer:

- Both are interfaces to execute code immediately after Spring Boot starts.

Feature	CommandLineRunner	ApplicationRunner
Arguments	<code>String[] args</code>	<code>ApplicationArguments args</code>
Use case	Simple startup tasks	Structured argument handling
Example	Insert initial DB data, logging	Parse CLI arguments like <code>--name=Satyam</code>

Example Code:

```
@Component
public class DataLoader implements CommandLineRunner {
    @Override
    public void run(String... args) {
        System.out.println("Inserting default data...");
    }
}

@Component
public class MyAppRunner implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) {
        System.out.println("App started with: " + args.getOptionValues("name"));
    }
}
```

```
}
```

Pro Tip:

- Use `ApplicationRunner` if you need **structured CLI arguments**.
- Multiple runners can exist, and their order can be controlled with `@Order`.

When asked “**Explain Spring Boot core concepts**”:

“Spring Boot simplifies application development with starter dependencies, opinionated setup, and auto-configuration. Starters reduce manual library setup, auto-configuration initializes beans based on classpath, and `application.properties/yml` externalizes configuration. `CommandLineRunner` and `ApplicationRunner` allow running code after startup. Embedded server and automatic JSON handling reduce boilerplate, making Spring Boot ideal for rapid development and microservices.”

If you want, I can **make a single visual flow diagram** showing:

Starters → Auto-Configuration → Beans → Controllers → JSON Response → Client

This will make your explanation **very visual and impressive in interviews**.

Do you want me to make that diagram?